

# TP1 — Introduction au neurone linéaire et à la descente de gradient

Machine Learning 2 — Deep Learning

## Objectifs du TP

Ce premier TP a pour objectif de comprendre les mécanismes fondamentaux utilisés pour entraîner un modèle de Machine Learning et, plus tard, un réseau de neurones.

À la fin de ce TP, vous serez capables de :

- comprendre le fonctionnement d'un neurone artificiel simple ;
- comprendre le rôle du poids et du biais ;
- comprendre comment un modèle produit une prédiction ;
- utiliser une fonction de perte pour mesurer l'erreur ;
- comprendre et implémenter la descente de gradient ;
- comprendre le rôle du learning rate ;
- comprendre l'intérêt de la standardisation ;
- séparer les données en ensembles d'entraînement et de validation ;
- utiliser les métriques MSE et MAE ;
- comparer la descente de gradient avec la solution analytique d'une régression linéaire.

**Librairies utilisées :** Python 3.9+, NumPy, Pandas, Matplotlib et scikit-learn.

## 1. Pourquoi commencer par un modèle aussi simple ?

Le Deep Learning utilise des réseaux de neurones qui peuvent contenir un très grand nombre de neurones et de paramètres.

Avant d'étudier ces modèles complexes, il est important de comprendre une question fondamentale :

### Comment un modèle apprend-il à partir de données ?

Pour répondre à cette question, nous allons commencer par le cas le plus simple possible : un seul neurone, une seule entrée et une seule sortie.

Prenons un exemple.

Nous souhaitons prédire la valeur d'un logement à partir du nombre moyen de pièces.

Nous avons donc :

**Entrée x** : nombre moyen de pièces

**Sortie y** : valeur réelle du logement

Le modèle reçoit x et produit une prédiction notée  $\hat{y}$  :

$$x \rightarrow \text{modèle} \rightarrow \hat{y}$$

Mais au début de l'apprentissage, le modèle ne sait pas encore comment produire une bonne prédiction.

Il doit apprendre à partir des exemples disponibles.

Pour comprendre cet apprentissage, nous devons répondre à trois questions.

### **1. Comment produire une prédiction ?**

Nous allons utiliser un modèle très simple :

$$\hat{y} = wx + b$$

### **2. Comment savoir si la prédiction est bonne ?**

Nous allons comparer la prédiction  $\hat{y}$  avec la vraie valeur y.

Une fonction de perte permettra de mesurer l'erreur du modèle.

### **3. Comment améliorer automatiquement le modèle ?**

Si la prédiction est mauvaise, nous devons modifier w et b pour réduire l'erreur.

C'est le rôle de la **descente de gradient**.

Le processus d'apprentissage peut donc être résumé ainsi :

**Données  $\rightarrow$  Prédiction  $\rightarrow$  Erreur  $\rightarrow$  Correction des paramètres  $\rightarrow$  Nouvelle prédiction**

Ce processus est répété plusieurs fois jusqu'à obtenir des prédictions satisfaisantes.

## **2. Quel est le lien avec le Deep Learning ?**

Dans ce TP, notre modèle est très simple :

$$x \rightarrow wx + b \rightarrow \hat{y}$$

Dans un réseau de neurones profond, nous aurons plusieurs entrées, plusieurs neurones et plusieurs couches :

**Entrées → Couche 1 → Couche 2 → ... → Sortie**

Le modèle sera beaucoup plus complexe, mais le principe d'apprentissage restera similaire :

1. recevoir des données ;
2. produire une prédiction ;
3. comparer la prédiction à la vraie valeur ;
4. calculer l'erreur ;
5. déterminer comment modifier les paramètres ;
6. modifier les paramètres ;
7. recommencer.

Les notions étudiées dans ce TP sont donc essentielles pour la suite du cours :

**Poids → Biais → Prédiction → Loss → Gradient → Learning rate → Mise à jour**

### 3. Qu'est-ce qu'un neurone artificiel ?

Un neurone artificiel est l'une des unités de base d'un réseau de neurones.

Dans notre cas, il reçoit une seule valeur  $x$ .

Il effectue le calcul suivant :

$$z = \mathbf{w}x + \mathbf{b}$$

où :

- $x$  est l'entrée ;
- $w$  est le poids (*weight*) ;
- $b$  est le biais (*bias*) ;
- $z$  est le résultat de la combinaison linéaire.

Dans un réseau de neurones, on applique généralement ensuite une fonction d'activation  $\phi$  :

$$\hat{y} = \phi(z)$$

Donc :

$$\hat{y} = \phi(\mathbf{w}x + \mathbf{b})$$

Dans ce premier TP, nous n'utilisons pas encore de fonction d'activation non linéaire. Nous utilisons simplement l'identité :

$$\phi(z) = z$$

Notre modèle devient donc :

$$\hat{y} = wx + b$$

Ce modèle correspond mathématiquement à une régression linéaire.

**Remarque :** le terme *perceptron* désigne historiquement un modèle de classification. Ici, nous travaillons plus précisément avec un **neurone linéaire**, afin d'introduire les mécanismes d'apprentissage qui seront ensuite utilisés dans les réseaux de neurones.

## 4. Comprendre le poids $w$

Considérons :

$$\hat{y} = wx + b$$

Le poids  $w$  contrôle l'influence de l'entrée  $x$  sur la prédiction.

Prenons :

$$w = 2$$

$$b = 1$$

Si :

$$x = 3$$

alors :

$$\hat{y} = 2 \times 3 + 1 = 7$$

Si maintenant :

$$x = 4$$

nous obtenons :

$$\hat{y} = 2 \times 4 + 1 = 9$$

Lorsque  $x$  augmente de 1, la prédiction augmente ici de 2.

Cette valeur correspond au poids  $w$ .

Graphiquement, le poids représente donc la **pente de la droite**.

Un poids positif signifie que la sortie tend à augmenter lorsque l'entrée augmente.

Un poids négatif signifie que la sortie tend à diminuer lorsque l'entrée augmente.

## 5. Comprendre le biais b

Considérons d'abord un modèle sans biais :

$$\hat{y} = wx$$

Si  $x = 0$ , alors :

$$\hat{y} = 0$$

La droite est donc obligée de passer par le point (0, 0).

Prenons maintenant :

$$\hat{y} = wx + b$$

Si :

$$w = 2$$

et :

$$b = 3$$

alors pour  $x = 0$  :

$$\hat{y} = 2 \times 0 + 3 = 3$$

Le biais permet donc de déplacer la droite verticalement.

On peut retenir :

**w = pente de la droite**

**b = valeur prédite lorsque  $x = 0$**

Le biais rend donc le modèle plus flexible.

## 6. Comment mesurer l'erreur ?

Supposons que la vraie valeur soit :

$$y = 5$$

et que notre modèle prédise :

$$\hat{y} = 7$$

L'erreur est :

$$\text{erreur} = \hat{y} - y$$

Donc :

$$\text{erreur} = 7 - 5 = 2$$

Prenons maintenant une autre observation :

$$y = 5$$

$$\hat{y} = 3$$

Nous obtenons :

$$\text{erreur} = 3 - 5 = -2$$

Si nous calculions simplement la moyenne des erreurs, les valeurs positives et négatives pourraient s'annuler.

Nous allons donc utiliser une fonction de perte.

## 7. La fonction de perte MSE

La MSE signifie **Mean Squared Error**, ou erreur quadratique moyenne.

Elle est calculée de la manière suivante :

$$\text{MSE} = (1/N) \times \text{somme des } (\hat{y}_i - y_i)^2$$

où N représente le nombre d'observations.

Le principe est simple.

Pour chaque observation :

1. calculer la différence entre la prédiction et la vraie valeur ;
2. mettre cette différence au carré ;
3. calculer la moyenne de toutes les erreurs au carré.

Par exemple :

Si l'erreur vaut 2 :

$$2^2 = 4$$

Si l'erreur vaut -2 :

$$(-2)^2 = 4$$

Les erreurs positives et négatives ne peuvent donc plus s'annuler.

Une MSE faible signifie que les prédictions sont globalement proches des vraies valeurs.

Une MSE élevée signifie que les prédictions sont éloignées des vraies valeurs.

Une MSE égale à 0 signifie que toutes les prédictions sont exactes.

Notre objectif est donc :

**Trouver les valeurs de  $w$  et  $b$  qui rendent la MSE aussi petite que possible.**

## 8. Comment le modèle apprend-il ?

Au début de l'apprentissage, nous pouvons choisir :

$$w = 0$$

$$b = 0$$

Le modèle :

$$\hat{y} = wx + b$$

produit alors :

$$\hat{y} = 0$$

pour toutes les observations.

Les prédictions seront évidemment mauvaises dans la plupart des cas.

Nous devons donc modifier progressivement  $w$  et  $b$ .

La question devient :

**Dans quelle direction faut-il modifier  $w$  et  $b$  pour réduire l'erreur ?**

C'est précisément le rôle du **gradient**.

## 9. La descente de gradient

La descente de gradient est une méthode permettant de modifier progressivement les paramètres d'un modèle afin de réduire sa fonction de perte.

On peut imaginer la fonction de perte comme une vallée.

Nous sommes quelque part sur la pente et nous voulons atteindre le point le plus bas.

Le gradient nous indique la direction dans laquelle la fonction augmente.

Pour diminuer la fonction, nous allons donc dans la direction opposée.

Pour le poids :

$$\mathbf{w} \leftarrow \mathbf{w} - \text{learning\_rate} \times \text{gradient\_w}$$

Pour le biais :

$$\mathbf{b} \leftarrow \mathbf{b} - \text{learning\_rate} \times \text{gradient\_b}$$

Dans notre cas :

$$\text{gradient\_w} = 2 \times \text{moyenne}((\hat{y} - y) \times x)$$

et :

$$\text{gradient\_b} = 2 \times \text{moyenne}(\hat{y} - y)$$

Ces deux valeurs indiquent comment modifier  $w$  et  $b$  pour réduire la MSE.

## 10. Une étape d'apprentissage

Une étape d'apprentissage peut donc être décomposée ainsi.

### Étape 1 — Prédiction

Le modèle calcule :

$$\hat{y} = \mathbf{w}\mathbf{x} + \mathbf{b}$$

### Étape 2 — Erreur

On calcule :

$$\text{erreur} = \hat{y} - y$$

### Étape 3 — Loss

On calcule :

$$\text{MSE} = \text{moyenne}(\text{erreur}^2)$$

### Étape 4 — Gradients

On calcule :

$$\text{gradient\_w} = 2 \times \text{moyenne}(\text{erreur} \times x)$$

$$\text{gradient\_b} = 2 \times \text{moyenne}(\text{erreur})$$

### Étape 5 — Mise à jour

On modifie :

$$w \leftarrow w - \text{learning\_rate} \times \text{gradient\_w}$$

$$b \leftarrow b - \text{learning\_rate} \times \text{gradient\_b}$$

Puis le modèle effectue de nouvelles prédictions avec les nouvelles valeurs de  $w$  et  $b$ .

Le processus recommence.

## 11. Qu'est-ce qu'une epoch ?

Une **epoch** correspond à un passage complet sur toutes les données d'entraînement.

Si nous entraînons le modèle pendant 300 epochs, le processus :

**Prédiction → Erreur → Gradient → Mise à jour**

est répété 300 fois sur l'ensemble des données dans notre implémentation de descente de gradient *batch*.

Au début, la perte devrait être relativement élevée.

Au fur et à mesure des epochs, elle devrait diminuer.

Lorsqu'elle ne diminue presque plus, cela signifie que le modèle s'est rapproché d'un minimum.

## 12. Le learning rate

Le **learning rate** contrôle la taille des modifications apportées à  $w$  et  $b$ .

Prenons :

$$w \leftarrow w - \text{learning\_rate} \times \text{gradient\_w}$$

Si le learning rate est très petit,  $w$  change très peu à chaque étape.

Si le learning rate est grand,  $w$  change beaucoup.

### Learning rate trop petit

Le modèle peut apprendre correctement, mais très lentement.

Il faudra beaucoup d'epochs.

### Learning rate adapté

La perte diminue relativement rapidement et se stabilise.

### Learning rate trop grand

Les modifications deviennent trop importantes.

Le modèle peut dépasser le minimum, repartir dans l'autre direction et osciller.

Dans certains cas, la perte peut même augmenter fortement : l'apprentissage diverge.

Dans ce TP, nous allons observer directement ces différents comportements.

## 13. Pourquoi standardiser les données ?

Les variables d'un dataset peuvent avoir des échelles très différentes.

Par exemple :

- âge : entre 18 et 80 ;
- surface : entre 20 et 300 ;
- revenu : entre 1 000 et 100 000.

Ces différences peuvent compliquer l'optimisation.

La standardisation transforme une valeur  $x$  de la manière suivante :

**$x_{\text{standardisé}} = (x - \text{moyenne}) / \text{écart-type}$**

Après standardisation :

**$\text{moyenne} \approx 0$**

et :

**$\text{écart-type} \approx 1$**

Cela permet de placer les données sur une échelle plus adaptée à l'optimisation.

La standardisation devient particulièrement importante lorsque plusieurs variables d'entrée ont des échelles très différentes.

## 14. Entraînement et validation

Nous ne devons pas uniquement vérifier si le modèle fonctionne sur les données qu'il a utilisées pour apprendre.

Nous allons donc séparer notre dataset en deux parties.

### **Training set**

L'ensemble d'entraînement est utilisé pour apprendre les paramètres :

**w et b**

### **Validation set**

L'ensemble de validation contient des observations qui ne sont pas utilisées pour ajuster directement w et b.

Nous pouvons par exemple utiliser :

**80 % des données pour l'entraînement**

et :

**20 % des données pour la validation**

Nous pouvons ensuite vérifier si le modèle produit également de bonnes prédictions sur les données de validation.

## 15. MSE et MAE

Nous allons utiliser deux métriques pour évaluer notre modèle.

### MSE — Mean Squared Error

$$\text{MSE} = \text{moyenne}((y - \hat{y})^2)$$

La MSE met les erreurs au carré.

Elle pénalise donc fortement les grandes erreurs.

### MAE — Mean Absolute Error

$$\text{MAE} = \text{moyenne}(|y - \hat{y}|)$$

La MAE utilise la valeur absolue de l'erreur.

Elle est généralement plus facile à interpréter car elle reste dans la même unité que la variable que nous cherchons à prédire.

## 16. Ce qu'il faut retenir avant de commencer le TP

Nous partons d'une entrée :

**x**

Le modèle produit :

$$\hat{y} = \mathbf{w}\mathbf{x} + \mathbf{b}$$

Nous comparons cette prédiction avec :

**y**

Nous calculons une erreur :

$$\text{erreur} = \hat{y} - y$$

Puis une loss :

$$\text{MSE} = \text{moyenne}(\text{erreur}^2)$$

Nous calculons ensuite les gradients.

Enfin, nous modifions  $w$  et  $b$  :

$w \leftarrow w - \text{learning\_rate} \times \text{gradient\_w}$

$b \leftarrow b - \text{learning\_rate} \times \text{gradient\_b}$

Et nous recommençons.

Le processus complet est donc :

**DONNÉES → PRÉDICTION → ERREUR → LOSS → GRADIENTS → MISE À JOUR → NOUVELLE PRÉDICTION**

C'est ce mécanisme que nous allons maintenant programmer.

# TP GUIDÉ

## Partie 1 — Chargement du dataset

Nous utiliserons le dataset California Housing.

Pour simplifier le problème, nous conservons une seule variable explicative :

- `AveRooms` : nombre moyen de pièces ;
- `PRICE` : valeur médiane des logements.

```
import numpy as np
import pandas as pd
import matplotlib.pyplot as plt

USE_SYNTHETIC = False
df = None

try:
    from sklearn.datasets import fetch_california_housing

    cal = fetch_california_housing(as_frame=True)
    df = cal.frame[['AveRooms', 'MedHouseVal']].copy()
    df.rename(columns={'MedHouseVal': 'PRICE'}, inplace=True)

except Exception:
    print("Dataset indisponible, génération d'un dataset synthétique.")
    USE_SYNTHETIC = True

if USE_SYNTHETIC:
    rng = np.random.default_rng(42)

    n = 1200
    AveRooms = rng.normal(5.5, 1.2, size=n)
```

```
PRICE = (
    0.8 * AveRooms
    + rng.normal(0.0, 0.5, size=n)
    + 1.5
)

df = pd.DataFrame({
    'AveRooms': AveRooms,
    'PRICE': PRICE
})

df.head()
```

## Travail demandé

1. Affichez cinq lignes du dataset.
2. Affichez sa taille.
3. Identifiez la variable d'entrée.
4. Identifiez la variable à prédire.

## Partie 2 — Exploration des données

```
plt.scatter(df['AveRooms'], df['PRICE'], alpha=0.4)

plt.xlabel("AveRooms")
plt.ylabel("PRICE")
plt.title("Relation AveRooms → PRICE")

plt.show()

print("Taille du dataset :", df.shape)
print(df.describe())
```

## Questions

1. Observez-vous une relation entre AveRooms et PRICE ?
2. La relation semble-t-elle parfaitement linéaire ?
3. Observez-vous une forte dispersion ?
4. Observez-vous des valeurs atypiques ?
5. Pensez-vous que le nombre de pièces suffit à expliquer complètement le prix d'un logement ? Pourquoi ?

## Partie 3 — Préparation et standardisation

```
X = df['AveRooms'].to_numpy().astype(float)
y = df['PRICE'].to_numpy().astype(float)

Xn = (X - X.mean()) / X.std()
```

```
print("Avant standardisation")
print("Moyenne :", X.mean())
print("Écart-type :", X.std())

print("\nAprès standardisation")
print("Moyenne :", Xn.mean())
print("Écart-type :", Xn.std())
```

## Questions

1. Quelle est la moyenne de  $X_n$  ?
2. Quel est son écart-type ?
3. Est-ce cohérent avec la définition de la standardisation ?

# Partie 4 — Premier modèle : sans biais

Nous commençons par :

$$\hat{y} = \mathbf{w}\mathbf{x}$$

Complétez le programme suivant.

```
def train_linear_no_bias(X, y, lr=0.05, epochs=200):

    w = 0.0
    losses = []

    for epoch in range(epochs):

        # 1. Prédiction
        yhat = ...

        # 2. Erreur
        error = ...

        # 3. MSE
        loss = ...

        # 4. Gradient de w
        grad_w = ...

        # 5. Mise à jour de w
        w = ...

        losses.append(loss)

    return w, np.array(losses)
```

## Rappel

Vous devez traduire en Python :

$$\hat{y} = wx$$

$$\text{erreur} = \hat{y} - y$$

$$\text{MSE} = \text{moyenne}(\text{erreur}^2)$$

$$\text{gradient\_w} = 2 \times \text{moyenne}(\text{erreur} \times x)$$

$$w \leftarrow w - \text{learning\_rate} \times \text{gradient\_w}$$

## Partie 5 — Observer l'apprentissage

```
w1, losses1 = train_linear_no_bias(  
    Xn,  
    Y,  
    lr=0.05,  
    epochs=200  
)  
  
print("Poids appris :", w1)  
  
plt.plot(losses1)  
plt.xlabel("Epoch")  
plt.ylabel("MSE")  
plt.title("Évolution de la perte")  
plt.show()
```

### Questions

1. La MSE diminue-t-elle ?
2. Que signifie cette diminution ?
3. Pourquoi la courbe finit-elle par devenir presque horizontale ?

## Partie 6 — Ajouter le biais

Nous utilisons maintenant :

$$\hat{y} = wx + b$$

Complétez :

```
def train_linear_with_bias(X, y, lr=0.05, epochs=300):  
    w = 0.0  
    b = 0.0  
    losses = []  
  
    for epoch in range(epochs):
```

```

        # Prédiction
        yhat = ...

        # Erreur
        error = ...

        # MSE
        loss = ...

        # Gradients
        grad_w = ...
        grad_b = ...

        # Mise à jour
        w = ...
        b = ...

        losses.append(loss)

    return w, b, np.array(losses)

```

## Rappel

**gradient\_w = 2 × moyenne(erreur × X)**

**gradient\_b = 2 × moyenne(erreur)**

# Partie 7 — Visualiser le modèle appris

Après avoir complété la fonction :

```

w2, b2, losses2 = train_linear_with_bias(
    Xn,
    y,
    lr=0.05,
    epochs=300
)

print("w =", w2)
print("b =", b2)

x_plot = np.linspace(Xn.min(), Xn.max(), 200)
y_plot = w2 * x_plot + b2

plt.scatter(Xn, y, alpha=0.3, label="Données")
plt.plot(x_plot, y_plot, linewidth=2, label="Modèle appris")

plt.xlabel("AveRooms standardisé")
plt.ylabel("PRICE")
plt.legend()
plt.show()

```

## Questions

1. La droite suit-elle la tendance générale des données ?
2. Pourquoi tous les points ne sont-ils pas sur la droite ?
3. Que représente la distance entre un point et la droite ?

## Partie 8 — Séparation entraînement / validation

```
from sklearn.model_selection import train_test_split
from sklearn.metrics import mean_squared_error, mean_absolute_error

Xtr, Xva, ytr, yva = train_test_split(
    Xn.reshape(-1, 1),
    Y,
    test_size=0.2,
    random_state=42
)

w3, b3, _ = train_linear_with_bias(
    Xtr.ravel(),
    ytr,
    lr=0.05,
    epochs=300
)

y_pred_val = w3 * Xva.ravel() + b3

mse = mean_squared_error(yva, y_pred_val)
mae = mean_absolute_error(yva, y_pred_val)

print(f"MSE validation = {mse:.4f}")
print(f"MAE validation = {mae:.4f}")
```

### Questions

1. Que représente la MSE obtenue ?
2. Que représente la MAE ?
3. Pourquoi évaluons-nous le modèle sur des données différentes de celles utilisées pour apprendre  $w$  et  $b$  ?

## Partie 9 — Influence du learning rate

Testez :

```
learning_rates = [0.0005, 0.005, 0.05, 0.2, 0.5]

for lr in learning_rates:
    _, _, losses = train_linear_with_bias(
```

```

        Xn,
        y,
        lr=lr,
        epochs=250
    )

    plt.plot(losses)
    plt.title(f"Learning rate = {lr}")
    plt.xlabel("Epoch")
    plt.ylabel("MSE")
    plt.show()

```

Complétez :

### Learning rate Convergence lente/rapide Stable ? Commentaire

0.0005

0.005

0.05

0.2

0.5

### Question

Expliquez avec vos propres mots pourquoi le learning rate influence la convergence.

## Partie 10 — Sans biais ou avec biais ?

Entraînez les deux modèles sur les données d'entraînement :

**Modèle 1 :  $\hat{y} = \mathbf{w}\mathbf{x}$**

**Modèle 2 :  $\hat{y} = \mathbf{w}\mathbf{x} + \mathbf{b}$**

Calculez ensuite MSE et MAE sur les données de validation.

Complétez :

**Modèle   MSE validation   MAE validation**

Sans biais

Avec biais

### Questions

1. Quel effet observez-vous après l'ajout du biais ?
2. Pourquoi le modèle sans biais est-il plus contraint ?
3. Que doit obligatoirement faire une droite sans biais lorsque  $x = 0$  ?

## Partie 11 — Effet de la standardisation

```
w_ns, b_ns, losses_ns = train_linear_with_bias(  
    X,  
    Y,  
    lr=0.0001,  
    epochs=400  
)  
  
w_s, b_s, losses_s = train_linear_with_bias(  
    Xn,  
    Y,  
    lr=0.05,  
    epochs=400  
)  
  
plt.plot(losses_ns, label="Sans standardisation")  
plt.plot(losses_s, label="Avec standardisation")  
  
plt.xlabel("Epoch")  
plt.ylabel("MSE")  
plt.title("Comparaison de la convergence")  
plt.legend()  
plt.show()
```

### Attention

Les deux modèles utilisent ici des learning rates différents.

Cette expérience permet principalement d'observer que l'échelle des données influence le comportement de la descente de gradient et les valeurs de learning rate utilisables.

### Questions

1. Comparez les courbes.
2. Pourquoi le même learning rate n'est-il pas nécessairement adapté dans les deux situations ?
3. Quel est l'intérêt de la standardisation ?

## Partie 12 — Comparaison avec la solution analytique

Dans le cas particulier d'une régression linéaire simple, il existe une formule permettant de calculer directement  $w$  et  $b$ .

Nous pouvons utiliser :

```

x_mean = Xn.mean()
y_mean = y.mean()

w_star = (
    ((Xn - x_mean) * (y - y_mean)).sum()
    / ((Xn - x_mean) ** 2).sum()
)

b_star = y_mean - w_star * x_mean

print(f"Solution analytique : w* = {w_star:.4f}, b* = {b_star:.4f}")
print(f"Descente de gradient : w = {w2:.4f}, b = {b2:.4f}")

```

Complétez :

|                      | Méthode | w b |
|----------------------|---------|-----|
| Descente de gradient |         |     |
| Solution analytique  |         |     |

## Questions

1. Les valeurs obtenues sont-elles proches ?
2. Pourquoi devraient-elles être proches ?
3. Si elles ne le sont pas, quelles pourraient être les causes ?

# Travail à rendre

Votre compte rendu doit contenir :

## 1. Exploration des données

La figure AveRooms → PRICE accompagnée d'une courte interprétation.

## 2. Influence du learning rate

Les courbes de perte pour au moins trois valeurs différentes et une explication des comportements observés.

## 3. Modèle sans biais / avec biais

Un tableau contenant la MSE et la MAE des deux modèles et une interprétation.

## 4. Standardisation

Une figure comparant la convergence avec et sans standardisation, accompagnée d'une explication.

## 5. Descente de gradient / solution analytique

Un tableau comparant les valeurs de  $w$  et  $b$  obtenues avec les deux méthodes.

## 6. Conclusion

Expliquez en quelques lignes ce que vous avez compris du mécanisme d'apprentissage.

Votre explication doit faire apparaître les notions suivantes :

**prédiction → erreur → fonction de perte → gradient → learning rate → mise à jour des paramètres**

# Consignes

Le TP doit être réalisé à partir du cours, des explications données pendant la séance, de la documentation des bibliothèques utilisées et de vos propres expérimentations.

**L'utilisation d'outils d'intelligence artificielle générative pour générer le code, répondre aux questions, interpréter les résultats ou rédiger le compte rendu n'est pas autorisée pour ce TP.**

Vous devez être capable d'expliquer le code que vous rendez ainsi que les résultats obtenus.

# À retenir

Un modèle reçoit une entrée  $x$ .

Il produit une prédiction :

$$\hat{y} = wx + b$$

Cette prédiction est comparée à la vraie valeur  $y$ .

Nous calculons ensuite une fonction de perte.

Les gradients indiquent comment modifier les paramètres.

Le learning rate détermine la taille de cette modification.

Le processus recommence jusqu'à ce que la perte ne diminue presque plus.

Le mécanisme essentiel est donc :

**ENTRÉES → PRÉDICTION → LOSS → GRADIENTS → MISE À JOUR DES PARAMÈTRES**

Dans ce TP, nous l'avons appliqué à un seul neurone.

Dans les prochains TPs, nous ajouterons plusieurs neurones, des fonctions d'activation et plusieurs couches. Le modèle deviendra plus complexe, mais le principe fondamental de l'apprentissage restera le même.